

Project Based Learning Report on

EMAIL ALERTS ON WHATSAPP

Submitted to
Department of Data Science
Bachelor of Technology
in
CSE-DS
(2022-2026)
GROUP NO:2

S. Laxmi Priya (22R11A6735)

P. Mythri (22R11A6731)

G. Rajeswari (22R11A6717)

Under the Guidance of:
Ms. M.V. LAVANYA, Asst.
professor.



Department of CSE-DS
Accredited by NBA
Geethanjali College of Engineering and Technology (UGC Autonomous)
(Affiliated to J.N.T.U.H, Approved by AICTE, New Delhi) Cheeryal
(V), Keesara (M), Medchal.Dist.-501 301.

JUNE-2024

TABLE OF CONTENTS

S.No	Index	Page No.
1.	ABSTRACT	3
2.	INTRODUCTION	4-5
3.	OBJECTIVE	6-8
4.	SOFTWARE & HARDWARE REQUIREMENTS	9
5.	MODULES	10-12
6.	CODE	13-16
7.	OUTPUT	17
8.	CONCLUSION	18
9.	REFERENCES	19

1.ABSTRACT

In the digital age, the seamless integration of various communication platforms is essential for efficient information dissemination. This project aims to bridge email and instant messaging by creating a system that sends email alerts directly to WhatsApp using Python. The solution leverages Python's capabilities to interact with email servers and the Twilio API for WhatsApp messaging, providing a practical and real-time notification system.

The project is divided into four primary stages:

1. Setting Up the Development Environment
2. Reading Emails Using Python
3. Sending Messages to WhatsApp Using Twilio
4. Combining Email Reading and WhatsApp Sending

2.INTRODUCTION

In an era where communication is instantaneous and ubiquitous, the ability to streamline and automate notifications across different platforms is increasingly valuable. Email remains a cornerstone of digital communication, widely used for both personal and professional correspondence. However, the immediacy and convenience of instant messaging platforms like WhatsApp have made them a preferred medium for real-time communication.

This project aims to develop a system that bridges the gap between email and instant messaging by sending email alerts directly to WhatsApp using Python. The motivation behind this project is to ensure that important emails do not go unnoticed, especially when quick action is required. By receiving email alerts on WhatsApp, users can stay informed and respond promptly without constantly checking their email inbox.

The core objectives of this project are:

- 1. Ease of Setup and Use:** To create a straightforward script that can be set up with minimal effort, making it accessible even for beginners.
- 2. Real-time Notification:** To ensure that email alerts are sent to WhatsApp in real-time, providing timely updates.
- 3. Scalability and Flexibility:** To build a system that can be easily extended with additional features, such as filtering specific types of emails or handling email attachments.

The implementation of this project involves leveraging Python's extensive libraries for email handling and Twilio's API for WhatsApp messaging. The process is divided into four main stages: setting up the development environment, reading emails, sending messages to WhatsApp, and combining these functionalities into a cohesive system.

By integrating email and WhatsApp notifications, this project addresses a practical need for improved communication efficiency. It demonstrates how software engineering can enhance everyday workflows, ensuring critical information is delivered through preferred communication channels.

3. OBJECTIVE

The primary objective of this project is to develop a robust and efficient system that automatically sends email alerts to WhatsApp, leveraging Python and the Twilio API. This system aims to enhance real-time communication and ensure important emails are promptly acknowledged and acted upon.

The specific objectives are as follows:

1. Automate Email Monitoring:

- Create a Python script that connects to an email server using the IMAP protocol, authenticates the user, and continuously monitors the inbox for new, unread emails.

2. Real-time Notification Delivery:

- Implement a mechanism to fetch email subjects and promptly send them as notifications to a specified WhatsApp number using the Twilio API.

3. User-friendly Setup:

- Ensure the system is easy to set up and configure, even for users with minimal programming experience. This includes clear documentation and straightforward configuration of email and Twilio credentials.

4. Reliability and Accuracy:

- Design the system to handle various types of email subjects, including those with special characters and different encodings, ensuring that messages are accurately delivered.

5. Scalability and Extendability:

- Provide a foundational system that can be easily extended with additional features, such as filtering emails based on specific criteria, handling attachments, or integrating with other messaging platforms.

6. Security and Privacy:

- Implement best practices for handling user credentials securely, ensuring that sensitive information such as email passwords and Twilio API keys are protected.

By achieving these objectives, the project will deliver a practical tool for bridging email communication with instant messaging, enhancing responsiveness and ensuring that critical information is always within reach.

4.SOFTWARE & HARDWARE REQUIREMENTS

Software Requirements:

1. **Python:** Version 3.x (e.g., Python 3.7 or Python 3.8)
 - Python is used to write the script that connects to the email server and sends messages to WhatsApp.
2. **Python Libraries:**
 - imaplib: This library provides access to IMAP (Internet Message Access Protocol) mailboxes.
 - email: Used for parsing email messages and extracting subject lines.
 - twilio: Provides integration with the Twilio API for sending messages to WhatsApp.
3. **Twilio Account:**
 - Sign up for a free Twilio account at [twilio.com](https://www.twilio.com).
 - Obtain your Twilio Account SID, Auth Token, and a Twilio phone number configured for WhatsApp messaging.
4. **Email Account:**
 - An active email account that supports IMAP access (e.g., Gmail). Make sure IMAP access is enabled for your account.

Hardware Requirements:

1. **Computer:**
 - Any modern computer capable of running Python should suffice. This includes desktops, laptops, or even a Raspberry Pi.
 - Operating System: Python scripts can run on Windows, macOS, or Linux distributions.
2. **Internet Connection:**
 - Required for accessing email servers (IMAP) and sending messages via the Twilio API.

5. MODULES

Required Python Modules/Libraries:

1. **imaplib:**

- Purpose: Allows Python programs to manage mailboxes over the IMAP protocol.
- Usage: Used to connect to an email server, authenticate, and fetch emails.

2. **email:**

- Purpose: Provides a facility for managing email messages, including parsing, composing, and representing email messages.
- Usage: Essential for parsing email content, extracting subjects, and other metadata.

3. **twilio:**

- Purpose: Official Python helper library for the Twilio API.
- Usage: Facilitates sending messages to WhatsApp using Twilio's Programmable Messaging API.

4. **getpass:**

- Purpose: Provides a secure way to prompt for a password in a terminal-based application.
- Usage: Useful if you want to prompt the user for their email password without displaying it on the screen.

5. **datetime:**

- Purpose: Supplies classes for manipulating dates and times.
- Usage: Helps in formatting timestamps or scheduling tasks based on time.

6.CODE

```
import os
import pickle
import base64
import email
import time
import requests
from email.header import decode_header
import imaplib
from twilio.rest import Client
from google.auth.transport.requests import Request
from google.oauth2.credentials import Credentials
from google_auth_oauthlib.flow import InstalledAppFlow
from googleapiclient.discovery import build
from dotenv import load_dotenv
import httplib2

# Load environment variables
load_dotenv()

# Set up environment variables
EMAIL_USER = os.getenv('EMAIL_USER')
EMAIL_PASSWORD = os.getenv('EMAIL_PASSWORD')
TWILIO_ACCOUNT_SID = os.getenv('TWILIO_ACCOUNT_SID')
TWILIO_AUTH_TOKEN = os.getenv('TWILIO_AUTH_TOKEN')
```

```
TWILIO_WHATSAPP_NUMBER =  
os.getenv('TWILIO_WHATSAPP_NUMBER')
```

```
PERSONAL_WHATSAPP_NUMBER =  
os.getenv('PERSONAL_WHATSAPP_NUMBER')
```

If modifying these SCOPES, delete the file token.json.

```
SCOPES = ['https://www.googleapis.com/auth/gmail.readonly']
```

```
def get_gmail_service():
```

```
    """Shows basic usage of the Gmail API.
```

```
    Lists the user's Gmail labels.
```

```
    """
```

```
    creds = None
```

```
    if os.path.exists('token.json'):
```

```
        with open('token.json', 'rb') as token:
```

```
            creds = Credentials.from_authorized_user_file('token.json', SCOPES)
```

```
    if not creds or not creds.valid:
```

```
        if creds and creds.expired and creds.refresh_token:
```

```
            creds.refresh(Request())
```

```
        else:
```

```
            flow = InstalledAppFlow.from_client_secrets_file('credentials.json',  
SCOPES)
```

```
            creds = flow.run_local_server(port=0)
```

```
            with open('token.json', 'wb') as token:
```

```
                token.write(creds.to_json().encode())
```

```
    service = build('gmail', 'v1', credentials=creds)
```

```
    return service
```

```

def parse_email_parts(parts):
    """Parse the parts of an email to extract the body content."""
    body = ""
    for part in parts:
        if part['mimeType'] == 'text/plain':
            body = base64.urlsafe_b64decode(part['body']['data']).decode('utf-8')
            break
        elif 'parts' in part:
            body = parse_email_parts(part['parts'])
            if body:
                break
    return body

def read_latest_email():
    service = get_gmail_service()
    max_retries = 5
    for attempt in range(max_retries):
        try:
            results = service.users().messages().list(userId='me',
maxResults=1).execute()
            message_id = results['messages'][0]['id']
            message = service.users().messages().get(userId='me',
id=message_id).execute()
            break
        except (requests.ConnectionError, requests.Timeout,
httplib2.ServerNotFoundError) as e:

```

```

        print(f"Network error occurred: {e}. Retrying
({attempt+1}/{max_retries})...")
        time.sleep(5)
    except Exception as e:
        print(f"An error occurred: {e}")
        raise
    else:
        raise Exception("Failed to connect to Gmail API after several attempts.")

    payload = message['payload']
    headers = payload.get('headers', [])
    subject = next((header['value'] for header in headers if header['name'] ==
'Subject'), None)
    if 'parts' in payload:
        body = parse_email_parts(payload['parts'])
    else:
        body = base64.urlsafe_b64decode(payload['body']['data']).decode('utf-8')
    return subject, body

def send_whatapp_message(to, body):
    client = Client(TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN)
    message = client.messages.create(
        body=body,
        from_=TWILIO_WHATSAPP_NUMBER,
        to=to
    )
    return message.sid

```

```
if _name_ == '_main_':
```

```
    subject, body = read_latest_email()
```

```
    print(f"Subject: {subject}")
```

```
    print(f"Body: {body}")
```

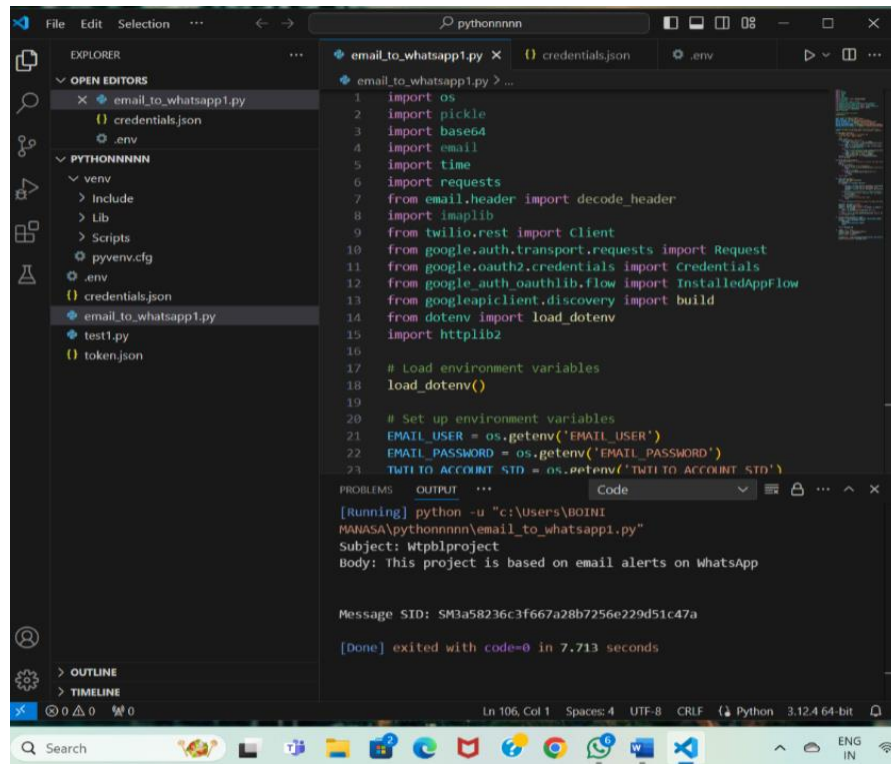
```
    message_sid
```

```
    send_whatsapp_message(PERSONAL_WHATSAPP_NUMBER, body)
```

```
    print(f"Message SID: {message_sid}")
```

=

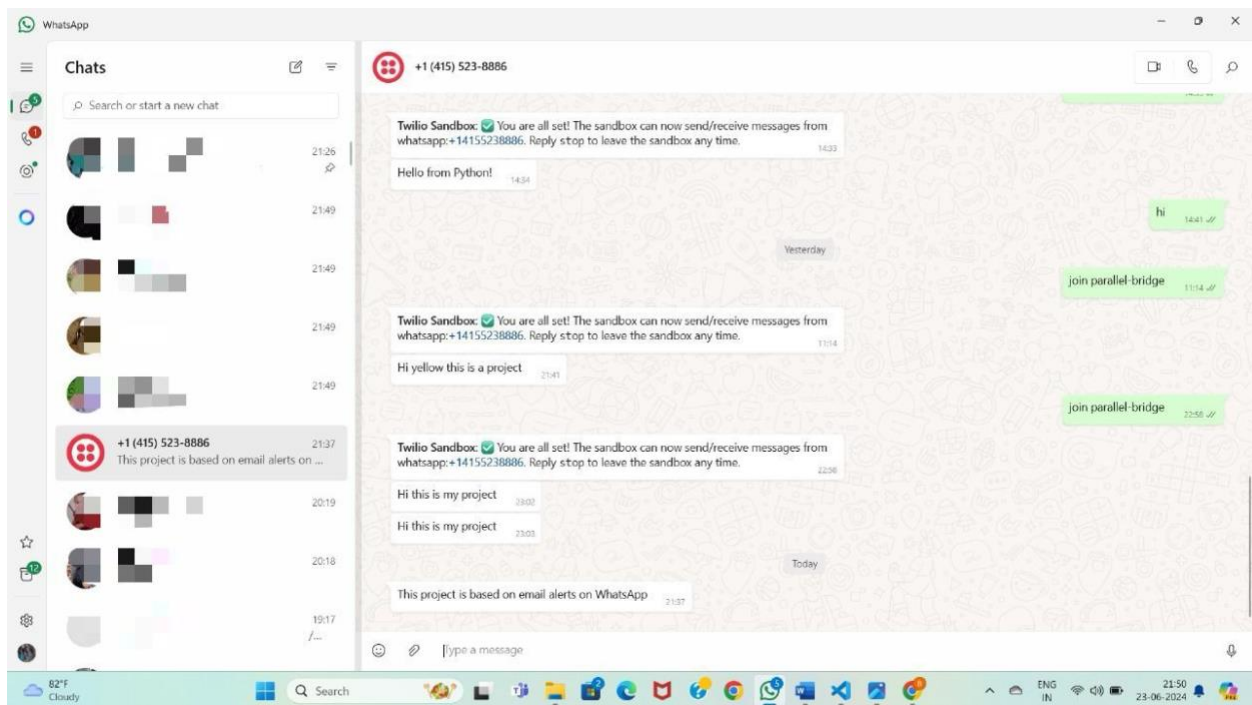
7.OUTPUT SCREEN



The screenshot shows a VS Code editor with a file explorer on the left and a code editor in the center. The file explorer shows a project named 'pythonnnnn' with files like 'email_to_whatsapp1.py', 'credentials.json', '.env', 'venv', 'Include', 'Lib', 'Scripts', 'pyvenv.cfg', 'test1.py', and 'token.json'. The code editor shows the content of 'email_to_whatsapp1.py', which is a Python script that imports various libraries and sets up environment variables. The output window at the bottom shows the execution of the script, which sends an email alert on WhatsApp.

```
1 import os
2 import pickle
3 import base64
4 import email
5 import time
6 import requests
7 from email.header import decode_header
8 import imaplib
9 from twilio.rest import Client
10 from google.auth.transport.requests import Request
11 from google.oauth2.credentials import Credentials
12 from google_auth_oauthlib.flow import InstalledAppFlow
13 from googleapiclient.discovery import build
14 from dotenv import load_dotenv
15 import httplib2
16
17 # Load environment variables
18 load_dotenv()
19
20 # Set up environment variables
21 EMAIL_USER = os.getenv('EMAIL_USER')
22 EMAIL_PASSWORD = os.getenv('EMAIL_PASSWORD')
23 TWILIO_ACCOUNT_SID = os.getenv('TWILIO_ACCOUNT_SID')
```

[Running] python -u "c:\Users\BOINI MANASA\pythonnnnn\email_to_whatsapp1.py"
Subject: Wtpblproject
Body: This project is based on email alerts on WhatsApp
Message SID: SM3a58236c3f667a28b7256e229d51c47a
[Done] exited with code=0 in 7.713 seconds



10 . CONCLUSION

In conclusion, the project exemplifies the power of Python and API integration in bridging disparate communication mediums for enhanced productivity and responsiveness. By automating email alerts to WhatsApp, we address the evolving demands of digital communication, setting a foundation for further innovation and adaptation in the realm of real-time notifications. This project not only meets immediate communication needs but also lays the groundwork for future enhancements and applications in the dynamic landscape of digital interaction.

11. REFERENCES

Access to link resource at:-

- Python Documentation:
 - Official Python documentation for imaplib: Python IMAP Library
- Twilio Documentation:
 - Twilio API documentation for sending WhatsApp messages: Twilio WhatsApp API
- Python Software Foundation. "IMAPClient Library Documentation." Accessed June 2024. Available at: <https://imapclient.readthedocs.io/en/3.1.1/>.
- Twilio. "Twilio API Documentation: WhatsApp API." Accessed June 2024. Available at: <https://www.twilio.com/docs/whatsapp/api>.

